

Un Sistema de Recuperación de Información con RAG para el Dominio de Tecnología y Software

Darío Francisco Alfonso Urrutia, Juan Carlos Pérez, and Sebastián González

Universidad de La Habana, Facultad de Matemática y Computación, Cuba

Resumen Este artículo presenta el diseño, implementación y evaluación de un Sistema de Recuperación de Información (SRI) especializado en el dominio de Tecnología y Software. El sistema integra la Indexación Semántica Latente (LSI) como modelo de recuperación central, complementado con un pipeline de Generación Aumentada por Recuperación (RAG) ejecutado sobre un modelo de lenguaje local. El corpus comprende 1 425 documentos provenientes de seis fuentes especializadas: Dev.to, HackerNews, RealPython, Lobsters, TheNewStack y TheVerge. El sistema alcanza una Precisión Media Promedio (MAP) de 0,6234 y una Tasa de Recíproco Promedio (MRR) de 0,7145. Se implementan y justifican módulos opcionales de evaluación de métricas y recomendación basada en contenido, junto con el despliegue reproducible mediante contenedores Docker.

Keywords: Recuperación de Información, Indexación Semántica Latente, Generación Aumentada por Recuperación, Rastreo Web, Procesamiento de Lenguaje Natural

1. Introducción

Los sistemas de recuperación de información ocupan un lugar central en los flujos de trabajo modernos de la ingeniería de software, donde los profesionales deben navegar volúmenes crecientes de documentación técnica, artículos y discusiones especializadas. El dominio de Tecnología y Software plantea desafíos particulares: el contenido evoluciona con rapidez, la terminología es altamente especializada, y las relaciones semánticas entre conceptos —como *contenedoriación*, *Docker* y *Kubernetes*— exigen modelos capaces de capturar significado latente más allá de la coincidencia exacta de términos.

El problema del *desfase de vocabulario* [2] es especialmente pronunciado en este dominio: un desarrollador que consulta “*machine learning*” debe encontrar documentos que traten sobre “*deep learning*”, “*redes neuronales*” o “*modelos de fundamento*”, pues estos conceptos comparten espacio semántico aunque difieran léxicamente. Los modelos booleanos y vectoriales clásicos son incapaces de capturar estas relaciones sin técnicas adicionales de normalización semántica.

Este trabajo describe la construcción de un SRI completo mediante: (1) un pipeline de rastreo multi-fuente que recopila contenido de seis plataformas técnicas especializadas; (2) un modelo de recuperación basado en LSI que descubre estructura temática latente mediante Descomposición en Valores Singulares

(DVS); (3) un módulo de Generación Aumentada por Recuperación que sintetiza respuestas a partir de los documentos recuperados empleando un LLM local; (4) un módulo de búsqueda web que se activa automáticamente ante insuficiencia local; (5) un motor de posicionamiento multi-señal; y (6) una interfaz visual construida con Gradio que expone el pipeline completo al usuario final.

La arquitectura adoptada sigue el principio de orquestación centralizada: todas las interacciones entre módulos pasan por la clase `MainOrchestator`, que actúa como única interfaz hacia los subsistemas. Esto garantiza independencia de la interfaz (CLI, API REST o GUI comparten el mismo backend), operaciones sin estado entre peticiones, y una estrategia de degradación elegante ante fallos parciales.

2. Trabajo Relacionado

Formalmente, un modelo de recuperación de información se define como un cuádruplo $[D, Q, F, R(q_j, d_i)]$ [1], donde D es el conjunto de representaciones lógicas de los documentos de la colección, Q es el conjunto de representaciones lógicas de las necesidades del usuario (consultas), F es el marco para modelar las representaciones de documentos y consultas y sus relaciones, y $R(q_j, d_i)$ es la función de *ranking* que asocia un número real a cada par consulta-documento, estableciendo un orden de relevancia. Esta definición es independiente del modelo concreto elegido y constituye el fundamento teórico sobre el que se construye el presente sistema.

LSI fue introducido por Deerwester et al. [2] como método para superar el problema del desfase de vocabulario proyectando documentos y consultas hacia un espacio latente de dimensionalidad reducida. La DVS truncada identifica los k ejes de máxima varianza en la matriz término-documento, agrupando automáticamente términos semánticamente relacionados. Este enfoque ha sido extendido hacia el LSI Probabilístico [3] y la Asignación Latente de Dirichlet [4], que introducen interpretaciones probabilísticas del espacio latente.

RAG fue propuesto por Lewis et al. [5] como marco para condicionar la generación de un modelo de lenguaje sobre documentos recuperados, reduciendo sustancialmente las alucinaciones al anclar las respuestas en fuentes verificables. La búsqueda web como mecanismo de respaldo ante corpus locales insuficientes sigue el paradigma de recuperación adaptativa en sistemas de respuesta a preguntas de dominio abierto [7].

Manning et al. [8] formalizan las métricas de evaluación estándar —MAP, MRR, NDCG— empleadas en este trabajo. El reranqueo neuronal mediante BERT [6] representa el estado del arte en posicionamiento, aunque su costo computacional lo hace impracticable para entornos sin GPU dedicada.

3. Definición del Dominio y Corpus

3.1. Dominio

El dominio seleccionado es **Tecnología y Software**, que comprende: marcos de trabajo y bibliotecas Python, DevOps y contenedorización (Docker, Kubernetes), aprendizaje automático y LLMs, desarrollo web, arquitectura de software, bases de datos, pipelines CI/CD, APIs y sistemas de recuperación de información. La selección responde a tres criterios: amplitud temática suficiente para generar un corpus diverso, disponibilidad de contenido técnico abierto de alta calidad, y relevancia directa para el contexto académico del equipo.

3.2. Estadísticas del Corpus

El corpus inicial fue construido mediante seis rastreadores especializados:

Cuadro 1. Composición del corpus por fuente

Fuente	Documentos	Cobertura
Dev.to	645	45,2 %
RealPython	725	50,9 %
Lobsters	19	1,3 %
HackerNews	10	0,7 %
TheNewStack	26	1,8 %
Total	1 425	100 %

Cada documento almacena: identificador UUID único, título, URL canónica, fecha de publicación en formato ISO 8601, texto completo, plataforma de origen, etiquetas temáticas, tipo de contenido y puntuación de popularidad. La longitud media de documento es de aproximadamente 274 tokens tras la normalización. El esquema unificado permite que todas las fuentes —incluyendo resultados de búsqueda web incorporados dinámicamente— sean procesadas de manera homogénea por el pipeline de indexación.

3.3. Proceso de Adquisición

El módulo de rastreo (`src/sri/crawler/`) implementa una jerarquía de clases con raíz en `BaseSpider`. La clase `ApiSpider` extiende la base para fuentes con API JSON, mientras que los rastreadores basados en RSS/Atom (`TheNewStackSpider`, `TheVergeSpider`) heredan directamente de `BaseSpider` y extraen el contenido completo embebido en los *feeds*. El rastreador de RealPython emplea el *sitemap* XML del sitio combinado con BeautifulSoup4 para la extracción de contenido estructurado.

Todos los rastreadores respetan el protocolo `robots.txt`, aplican limitación de velocidad configurable y almacenan los documentos en bruto como archivos

JSON en `data/raw/`. La clase `CrawlerCaller` orquesta la ejecución paralela de los seis rastreadores y consolida la salida en `data/documents.json`, aplicando deduplicación por hash de contenido.

4. Arquitectura del Sistema

El sistema se estructura en torno a la clase `MainOrchestator`, que coordina siete módulos independientes a través de un pipeline unificado de ejecución de consultas: (1) verificación del estado de la base de datos, (2) carga automática desde los rastreadores si la base está vacía, (3) búsqueda local multi-estrategia mediante `SRIPipeline`, (4) detección de insuficiencia por tres criterios, (5) búsqueda web condicional mediante `DuckDuckGo`, (6) deduplicación de documentos por hash de contenido, y (7) generación de respuesta RAG con extracción de citas.

```
query()
+> retrieve_documents()
|   +> check_database_health()
|   +> _search_locally() [LSI + TF-IDF + Vector]
|   +> detect_insufficiency()
|   +> (condicional) _search_web()
+> augment_response()
    +> RAGModule.generate()
    +> CitationExtractor.extract()
```

Figura 1. Flujo de ejecución del pipeline de consulta completo

Todos los módulos devuelven objetos Python estructurados, haciendo al orquestador agnóstico respecto a la interfaz: CLI, API REST y GUI comparten el mismo backend sin modificación. La estrategia de degradación elegante garantiza que un fallo en cualquier módulo —LLM no disponible, búsqueda web fallida— no interrumpa la respuesta al usuario, sino que retorne los resultados disponibles con metadatos del error.

5. Implementación de Módulos

5.1. Indexación

El indexador (`src/indexing/indexer.py`) construye un índice invertido con ponderación TF-IDF. La normalización textual incluye conversión a minúsculas, eliminación de puntuación, filtrado de palabras vacías en inglés y español, y derivación morfológica (*stemming*) mediante NLTK. La fórmula de ponderación TF-IDF empleada es:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)}$$

donde N es el número total de documentos y $\text{df}(t)$ el número de documentos que contienen el término t . El índice soporta consultas incrementales mediante el método `add_document()`, que actualiza las listas de posteo sin necesidad de reindexar el corpus completo.

5.2. Recuperación: Modelo LSI

El módulo de recuperación (`src/retrieval/lsi_model.py`) implementa LSI usando `TfidfVectorizer` y `TruncatedSVD` de scikit-learn [9]. El proceso de indexación sigue cuatro pasos:

1. Construir la matriz TF-IDF $M \in \mathbb{R}^{n \times v}$, donde n es el número de documentos y v el tamaño del vocabulario.
2. Aplicar DVS truncada: $M \approx U\Sigma V^T$, reteniendo $k = 100$ componentes latentes.
3. Normalizar $D = U\Sigma$ con normalización L2 para permitir similitud coseno mediante producto punto.
4. En tiempo de consulta: transformar mediante el vectorizador y la DVS ajustados, y calcular similitud coseno contra D .

Con 1 425 documentos indexados, LSI alcanza un 28,24% de varianza explicada en $k = 100$. La elección de LSI sobre modelos booleanos o TF-IDF puro responde a su capacidad de resolver sinonimia y polisemia —crítica en un dominio donde “*machine learning*” y “*aprendizaje profundo*” comparten espacio semántico latente, mientras que “*Python*” (lenguaje) y “*Python*” (serpiente) deben distinguirse por contexto.

5.3. Base de Datos Vectorial

El almacén vectorial (`src/retrieval/vector_store.py`) gestiona la búsqueda por similitud basada en *embeddings* mediante *sentence-transformers* con el modelo `all-MiniLM-L6-v2` (384 dimensiones). Soporta dos backends intercambiables siguiendo el principio Abierto/Cerrado: en memoria (por defecto) y ChromaDB para persistencia. La detección automática del backend disponible se realiza al inicializar el módulo, sin requerir cambios en el código cliente.

La huella de memoria típica para 1 425 documentos es de aproximadamente 50 MB para los vectores de *embedding* ($1\,425 \times 384 \times 4$ bytes), más el modelo LSI serializado (3 MB) y el índice invertido (2 MB). El LLM Ollama requiere adicionalmente 4 GB de RAM para el modelo `llama3.2` cuantizado.

5.4. Módulo RAG

El módulo RAG (`src/rag/`) conecta el pipeline de recuperación con Ollama (Llama 3.2). Incluye una fábrica de plantillas de *prompt* (`PromptTemplateFactory`) con tres variantes: básica, específica del dominio y con razonamiento en cadena. El *prompt* específico del dominio instruye al modelo a citar fuentes para cada afirmación, reconocer limitaciones informativas y rechazar explícitamente preguntas fuera del contexto técnico.

Se aplican cinco mecanismos anti-alucinación: (1) límite de longitud de contexto (`max_doc_content_length=1000` tokens por documento), (2) validación de citas mediante `CitationExtractor`, (3) control de temperatura ($T = 0,7$), (4) límite de tokens de salida (1024), e (5) instrucciones explícitas de rechazo para preguntas fuera de contexto. El analizador de salida (`OutputParser`) valida la estructura de la respuesta mediante Pydantic antes de exponerla al usuario.

5.5. Posicionamiento Multi-señal

El módulo de posicionamiento (`src/ranking/ranking.py`) implementa una función de puntuación multi-señal:

$$\text{score}(q, d) = 0,55 \cdot s_{\text{LSI}} + 0,25 \cdot s_{\text{vector}} + 0,10 \cdot s_{\text{frescura}} + 0,10 \cdot s_{\text{popularidad}}$$

La señal de frescura penaliza documentos con más de 180 días de antigüedad mediante una función de decaimiento exponencial. La señal de popularidad normaliza las puntuaciones de interacción de cada plataforma fuente (reacciones en Dev.to, vistas en RealPython). Se aplica un multiplicador de impulso para contenido de tipo tutorial o documentación, dado que en el dominio técnico este tipo de contenido presenta mayor densidad informativa por unidad de longitud. Los resultados locales tienen prioridad sobre los web-aumentados en la presentación final.

5.6. Búsqueda Web

El módulo de búsqueda web (`src/sri/web_search/`) emplea DuckDuckGo mediante `duckduckgo-search`. La insuficiencia se declara cuando se cumple al menos uno de tres criterios: (1) número de resultados < 3 , (2) puntuación media $< 0,55$, o (3) solape semántico insuficiente entre la consulta y los documentos recuperados, detectado mediante solapamiento de palabras clave tras eliminación de palabras vacías. Los resultados web se normalizan al esquema de documento unificado y se persisten en `data/documents.json` para sesiones futuras, enriqueciendo progresivamente el corpus local.

5.7. Módulo de Recomendación

El módulo de recomendación (`src/recommendation/`) implementa un sistema de recomendación híbrido que combina similitud de contenido con historial

persistente de búsqueda del usuario. Consta de dos componentes principales: `ContentBasedRecommender` y `UserSearchHistory`.

`ContentBasedRecommender` implementa la recomendación basada en contenido mediante una función de puntuación de tres señales:

$$\text{score}(d) = w_c \cdot s_{\text{contenido}} + w_r \cdot s_{\text{frescura}} + w_s \cdot s_{\text{fuente}}$$

con pesos por defecto $w_c = 0,82$, $w_r = 0,12$ y $w_s = 0,06$, configurables en tiempo de ejecución. La señal de contenido es la similitud coseno entre un vector de perfil —construido a partir de la consulta actual, intereses declarados y el centroide de los documentos seleccionados previamente— y la matriz TF-IDF del corpus (bigramas, `sublinear_tf`, hasta 50 000 características). La señal de frescura aplica decaimiento exponencial con semivida de 180 días. La señal de fuente asigna una *prior* proporcional al volumen de contenido de cada plataforma. El arranque en frío —sin perfil disponible— se resuelve degradando graciosamente a un ranking por frescura y *prior* de fuente.

`UserSearchHistory` persiste el historial de búsqueda en `data/user_history.json`. Cada evento de búsqueda almacena la consulta, la marca temporal y los identificadores de documentos recuperados. El método `build_profile()` construye automáticamente un perfil de recomendación a partir de las últimas n búsquedas (por defecto $n = 5$), concatenando los textos de consulta y los IDs de documentos recuperados como semillas para el recomendador de contenido. Esto permite que las recomendaciones se personalicen progresivamente conforme el usuario interactúa con el sistema, sin requerir registro explícito de preferencias.

La interfaz visual expone una pestaña de Recomendación con tres modos: recomendación manual por intereses, recomendación por similitud a un documento concreto, y recomendación automática basada en historial reciente. Tras cada búsqueda exitosa en la pestaña de Búsqueda, el sistema registra el evento y refresca automáticamente las recomendaciones automáticas.

5.8. Módulo de Evaluación

El módulo de evaluación (`src/evaluation/evaluation.py`) implementa las métricas estándar de SRI [8]: Precisión@k, Recall@k, F1@k, Precisión Media Promedio (MAP), Ganancia Acumulada Descontada Normalizada (NDCG@k) y Tasa de Recíproco Promedio (MRR). Acepta una especificación JSON de consultas de prueba con juicios de relevancia binarios o graduados, y produce métricas agregadas y por consulta.

La clase `Evaluator` expone un método `evaluate_all()` que acepta cualquier función de recuperación con la firma `Callable[[str], List[str]]`, desacoplando la evaluación del pipeline concreto. Esto permite comparar configuraciones alternativas del sistema —por ejemplo, con y sin expansión de consultas— sin modificar el código de evaluación.

5.9. Interfaz Visual

La interfaz Gradio (ui/) expone cuatro pestañas: Búsqueda, Configuración, Evaluación y Estado del Sistema. La pestaña de Búsqueda presenta los resultados posicionados con tarjetas de resultado que muestran título, fuente, puntuación y fragmento relevante, seguidos del panel RAG con la respuesta sintetizada y sus citas. La pestaña de Configuración agrupa los parámetros por ámbito de preocupación: parámetros de consulta, configuración RAG, configuración del rastreador y gestión de base de datos. Los cambios de configuración se propagan al pipeline a través de un objeto de estado compartido (`app_state`), sin requerir reinicio de la aplicación.

5.10. Despliegue: Docker

El sistema incluye un `Dockerfile` y un archivo `docker-compose.yml` que permiten reproducir el entorno de ejecución en cualquier máquina compatible con Docker, sin dependencias del sistema operativo anfitrión [11]. La imagen base `python:3.12-slim` minimiza el tamaño del contenedor; las dependencias se extraen automáticamente desde `pyproject.toml` mediante `tomllib` (biblioteca estándar desde Python 3.11), eliminando la necesidad de mantener un `requirements.txt` separado.

El archivo `docker-compose.yml` define dos servicios: `crawler`, que ejecuta el pipeline de adquisición de datos, y `demo`, que lanza el sistema completo. Ambos montan el directorio `./data` como volumen, preservando el corpus indexado entre ejecuciones y evitando la reindexación completa en cada arranque. Los pasos de despliegue son:

1. `docker compose build` — construye la imagen con todas las dependencias.
2. `docker compose run crawler` — recopila documentos del dominio.
3. `docker compose run demo` — ejecuta el sistema completo.

6. Justificación de los Módulos Opcionales

Módulo de Evaluación: MAP, MRR y NDCG ofrecen vistas complementarias de la calidad del posicionamiento, permitiendo la medición fundamentada de cambios de parámetros —por ejemplo, el valor de k en LSI o los pesos de la función de posicionamiento. La disponibilidad de métricas cuantitativas es imprescindible para guiar mejoras iterativas del sistema y comparar configuraciones de manera reproducible.

Módulo de Recomendación: En el dominio de Tecnología y Software, los usuarios frecuentemente necesitan explorar conceptos relacionados más allá de su consulta inicial. El sistema de recomendación basada en contenido permite sugerir artículos complementarios utilizando el historial de documentos seleccionados y el perfil de intereses del usuario, enriqueciendo la experiencia de recuperación sin requerir datos de comportamiento de otros usuarios —apropiado para un sistema con corpus cerrado.

Búsqueda Web: El corpus local envejece inevitablemente en un dominio de evolución rápida; nuevas versiones de frameworks, actualizaciones de seguridad y anuncios de herramientas pueden no estar cubiertos. La activación automática garantiza que las consultas sobre contenido reciente reciban información vigente, manteniendo la utilidad del sistema a lo largo del tiempo.

Posicionamiento Multi-señal: Las puntuaciones de relevancia simple no capturan la autoridad del documento. Un tutorial de RealPython con 10 000 visualizaciones es estadísticamente más probable de ser informativo que una publicación obscura con puntuación LSI similar. Las señales de frescura y popularidad mejoran la calidad del posicionamiento de manera complementaria a la relevancia semántica.

7. Resultados de Evaluación

Cuadro 2. Métricas de evaluación sobre el conjunto de prueba ($n = 5$ consultas)

Métrica	Valor
MAP (Precisión Media Promedio)	0,6234
MRR (Tasa de Recíproco Promedio)	0,7145
NDCG@5	0,6521
P@5 (Precisión a 5)	0,52
R@10 (Recall a 10)	0,675

Un MRR de 0,7145 indica que el primer documento relevante aparece en promedio en la posición 1,4. El MAP de 0,6234 confirma que los documentos relevantes se concentran en las posiciones superiores del ranking. La P@5 de 0,52 implica que aproximadamente 2,6 de los 5 primeros resultados son relevantes para la consulta media, valor competitivo considerando el tamaño reducido del conjunto de prueba y la naturaleza subjetiva de los juicios de relevancia en el dominio técnico.

Es importante señalar que el conjunto de prueba comprende únicamente 5 consultas con juicios de relevancia parciales, lo que limita la significancia estadística de las métricas. La construcción de un conjunto de evaluación más amplio con juicios de relevancia manuales representa la mejora metodológica prioritaria para trabajo futuro.

8. Análisis Crítico

8.1. Fortalezas

La arquitectura modular con interfaz limpia mediante `MainOrchestator` facilita el desarrollo y prueba independiente de cada módulo. El almacén vectorial de doble backend proporciona degradación elegante cuando ChromaDB no está

disponible. Los mecanismos anti-alucinación producen respuestas factuales fundamentadas, críticas en un dominio técnico donde la información incorrecta tiene consecuencias prácticas. El rastreo ético —cumplimiento de `robots.txt` y limitación de velocidad— garantiza una adquisición de datos sostenible y respetuosa con las fuentes.

La separación entre recuperación y generación, característica del paradigma RAG, permite actualizar el modelo de lenguaje sin reindexar el corpus, y viceversa. La persistencia automática de resultados web enriquece progresivamente el corpus local, reduciendo la latencia de futuras consultas similares.

8.2. Insuficiencias y Soluciones Propuestas

- **Desequilibrio del corpus** (RealPython: 50,9 %): Incorporar rastreadores para Go Blog, Rust Blog y plataformas orientadas a Java para diversificar las fuentes.
- **Hiperparámetro $k = 100$ de LSI sin ajustar**: Aplicar optimización bayesiana sobre $k \in \{50, 100, 200, 300\}$ maximizando MAP en el conjunto de validación.
- **Ausencia de recuperación híbrida**: Implementar Fusión de Ranking Recíproco (RRF) entre LSI y BM25 para combinar las fortalezas de ambos modelos.
- **Dependencia de Ollama en despliegue**: Empaquetar como imagen Docker autocontenida con Ollama incluido, eliminando la necesidad de instalación manual.
- **Verdad de base sintética en evaluación**: Realizar juicios de relevancia manuales sobre una muestra representativa de consultas para aumentar la validez de las métricas.
- **Sin caché de consultas**: Implementar caché LRU para consultas frecuentes, reduciendo la latencia de respuesta en un 60–80 % para patrones de uso repetitivo.

9. Conclusiones

Este artículo presentó un SRI completo para el dominio de Tecnología y Software que integra recuperación semántica basada en LSI, búsqueda web de respaldo, posicionamiento multi-señal y un pipeline RAG con LLM local. El sistema alcanza métricas competitivas (MAP 0,6234, MRR 0,7145) y ofrece una arquitectura modular limpia que soporta extensión futura. La implementación de dos módulos opcionales —evaluación cuantitativa y recomendación basada en contenido— demuestra la capacidad del sistema para abordar los desafíos propios del dominio seleccionado. Las deficiencias identificadas constituyen un mapa de ruta concreto para mejoras iterativas, con prioridad en la ampliación del conjunto de evaluación y el ajuste de hiperparámetros del modelo LSI.

Referencias

1. Baeza-Yates, R., Ribeiro-Neto, B.: Modern Information Retrieval. ACM Press / Addison-Wesley, Nueva York (1999)
2. Deerwester, S., Dumais, S.T., Furnas, G.W., Landauer, T.K., Harshman, R.: Indexing by latent semantic analysis. *J. Am. Soc. Inf. Sci.* 41(6), 391–407 (1990)
3. Hofmann, T.: Probabilistic latent semantic indexing. En: *Actas de SIGIR 1999*, pp. 50–57. ACM, Nueva York (1999)
4. Blei, D.M., Ng, A.Y., Jordan, M.I.: Latent Dirichlet allocation. *J. Mach. Learn. Res.* 3, 993–1022 (2003)
5. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. En: *Advances in NeurIPS*, vol. 33, pp. 9459–9474 (2020)
6. Nogueira, R., Cho, K.: Passage re-ranking with BERT. *arXiv:1901.04085* (2019)
7. Karpukhin, V., et al.: Dense passage retrieval for open-domain question answering. En: *Actas de EMNLP 2020*, pp. 6769–6781 (2020)
8. Manning, C.D., Raghavan, P., Schütze, H.: *Introduction to Information Retrieval*. Cambridge University Press (2008)
9. Pedregosa, F., et al.: Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.* 12, 2825–2830 (2011)
10. Abid, A., et al.: Gradio: Hassle-free sharing and testing of ML models in the wild. *arXiv:2106.05432* (2021)
11. Merkel, D.: Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal* 239, 2 (2014)